A Demonstration of Affordable Surface Scanning Technology

Levi C. Smith
Department of Physics, Portland State University, Portland, USA

(Dated: 4 June 2026)

Advances in technology in the virtual three-dimensional space have provoked the need for affordable options for the broader market. This paper demonstrates that an affordable 3D surface scanning setup can be created for less than $100 that is capable of sub-millimeter resolution. This is achieved through laser triangulation, the use of open source python libraries such as OpenCV to capture and process the images and the Poisson Surface Reconstruction algorithm to convert the scanned 3D point cloud into a watertight mesh. While analysis of the resulting mesh after scanning a sample surface indicated smoothing artifacts due to gaps in the points cloud via laser clipping, the artifacts were less than half a millimeter in size and did not affect the resultant mesh in any significant manner.

I. INTRODUCTION

Advances in modeling and manufacturing technology specifically in the three-dimensional space have provoked the need for accurate and fast 3D scanning technologies. Unfortunately, due to the majority need for industrial capable scanners, a need for affordable scanning systems with sufficient accuracy has arisen for applications including reverse engineering, proportional measurements, and other use cases that may not require the high precision of modern industrial scanners. The following report describes the design and demonstration of a 3D scanning system capable of sub-millimeter resolution that can be created for less than $100 using basic laser triangulation and image processing.

II. LASER TRIANGULATION & THEORY

To allow for the laser triangulation framework, the Pinhole Camera Model is used. This model simplifies the mathematical operations by assuming that light travels in straight lines and that all light rays detected by the camera through the camera lens pass through a single optical center. By this, the digital focal point of the used web camera can be used to map the two-dimensional image taken by the camera to the three-dimensional intersection of the light ray and the linear plane created by the laser line.

Let two coordinate systems be defined with the first coordinate system's origin places at the center of the camera lens and the second at the center of the laser lens. Let the point $P = (X_c, Y_c, Z_c)$ be a point in the camera's coordinate system where the light from the laser reflects off of it towards the camera's pinhole. A similar triangle can be formed as seen in Figure 1 such that $X_c/Z_c = x/f_x$, where f_x is the focal length along the y-axis in pixels and x is the distance in pixels from the center of the captured image. Thus solving for x results in

$$x = f_x \frac{X_c}{Z_c}. \tag{1}$$

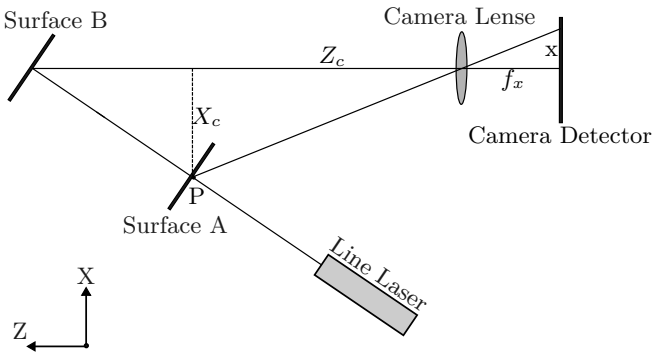

FIG. 1. A two-dimensional representation of laser triangulation using similar triangles with laser reflecting off of two surfaces. The triangle formed on the left-hand side of the camera lens is a similar triangle to the one formed on the right side of the lens with $X_c/Z_c = x/f_x$.

Similarly, considering the simplified yz-plane

$$y = f_y \frac{Y_c}{Z_c}. \tag{2}$$

While x and y describe how many pixels away from the optical center on the sensor the point is located, standard image coordinate systems start with the coordinate $(0, 0)$ in the top left corner of the image with u increasing to the right and v increasing downward. To account for this, the center coordinates can be added to each side of the equations above resulting in the standard coordinate equations of

$$u = x + c_u = f_x \frac{X_c}{Z_c} + c_u \tag{3}$$

and

$$v = y + c_v = f_y \frac{Y_c}{Z_c} + c_v, \tag{4}$$

where c_u and c_v are the center coordinates in the x-axis and y-axis respectively.

As seen in the laser-camera orientation described in Figure 2, the coordinate system as viewed by the camera

differs from the coordinate system of the laser, which is used as the real world coordinate system. To account for this difference, both a translation and rotation operation must be preformed on the coordinate system to adjust for the angle and height difference of the sensor. Thus,

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = R \cdot (P_{world} - T), \tag{5}$$

where R and T are the rotation and translation vectors respectively. Defining H to be the difference in height and θ to be the angle of orientation of the camera relative to the trajectory of the laser as seen in Figure 2, the translation and rotation vectors are

$$T = \begin{bmatrix} 0 \\ 0 \\ H \end{bmatrix} \tag{6}$$

and

$$R = \begin{bmatrix} 0 & -1 & 0 \\ -\sin\theta & 0 & -\cos\theta \\ \cos\theta & 0 & -\sin\theta \end{bmatrix} \tag{7}$$

respectively. Expanding the matrix multiplication yields the vector equality

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \begin{bmatrix} -y \\ -x\sin\theta + (H-z)\cos\theta \\ x\cos\theta + (H-z)\sin\theta \end{bmatrix} \tag{8}$$

which can be interpreted as the following system of linear equations:

$$X_c = -y \tag{9}$$

$$Y_c = -x\sin\theta + (H-z)\cos\theta \tag{10}$$

$$Z_c = x\cos\theta + (H-z)\sin\theta \tag{11}$$

Assuming these are plugged into Equation 3 and Equation 4, this results in two equations for the pixel coordinates with the three unknown world position variables x, y, and z. However, recall that this is the coordinate system relative to the laser head, and thus z will always be zero as the laser is mounted to the camera assembly and moves along with it. As shall be seen in later sections, the world reference z_w location is always known by design as the vertical location of the scanner assembly is known. Thus preforming the substitution with $z = 0$ results in

$$u = f_x \frac{-y}{x\cos\theta + (H)\sin\theta} - c_u \tag{12}$$

and

$$v = f_y \frac{-x\sin\theta + (H)\cos\theta}{x\cos\theta + (H)\sin\theta} - c_v \tag{13}$$

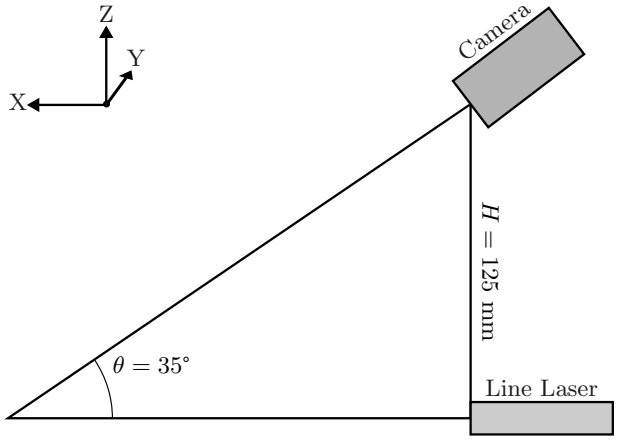

FIG. 2. The orientation of the camera to the laser where the center of the camera lens is fixed $H = 125$ mm above the center of the laser lens and is angled down from the horizontal toward the projection of the laser by $\theta = 35°$.

for the x and y pixel positions on the detector accordingly. Solving these equations for their real world positions yields

$$x = (H)\frac{f_y\cos\theta - (v - c_v)\sin\theta}{f_y\sin\theta + (v - c_v)\cos\theta} \tag{14}$$

and

$$y = -\frac{f_y}{f_x}\frac{(H)(u - c_u)}{f_y\sin\theta + (v - c_v)\cos\theta}. \tag{15}$$

III. SCANNER DESIGN

To ensure precise alignment and a solid fixture for the scanning assembly, the 3D modeling software Fusion 360 was used to model the scanning assembly with the constraints indicated in Figure 2. Additionally, it was designed to attach to a repurposed Photon Mono X 6K printer which served as the vertical linear actuator for the system.

Because traditional inexpensive lenses for laser lines output a Gaussian beam, meaning that the beam is wider in the center and tapers out towards the end, it was necessary to find a beam with uniform output to ensure an accurate reading of the laser line. While many industrial scanning system use these lasers, the priority of this study was to create a scanner for low cost. Thus, an affordable line laser with a Powell lens was purchased for $70. For the camera component, a used Logitech C920 Webcam was used as it provided a high 1080p resolution at an affordable price of $20 used.

These components were then used to construct the scanner assembly which was mounted to the repurposed Photon Mono X 6K printer. The built-in stepper motor and limit switch of the former printer were then connected to a TMC2209 stepper motor driver connected to

an Arduino Uno microcontroller. The microcontroller was programmed to precisely adjust the z-axis of the scanning head allowing for precise knowledge of the vertical location on the surface when preforming scans. The microcontroller was connected to the main processing application via a serial connection over a physical cable between the two devices. Allowing for the python program to control and zero vertical movement and position.

Given that an equivalent microcontroller and stepper motor driver can be found for less than $4 apiece, the total cost of the setup was $98. However, account for the stepper motor, lead screw and limit switch that may not be freely available from scrap, the total cost is likely closer to $130.

IV. IMAGE PROCESSING

To effectively preform laser triangulation an accurate reading of the position of the line at each pixel column must be identified for every scanned height increment. To accomplish this, the camera's brightness, contrast, sharpness, and saturation settings were tuned to isolate the laser line as much as possible from the rest of the capture. The computer graphics library OpenCV was then utilized to capture 10 samples of the image at each height. These images were passed through an HSV color filter to eliminate all remaining background noise leaving only the laser intensities on the image frame. The laser line coordinates on the frame for each column were then estimated with subpixel precision via a weighted grayscale average of the pixel intensities in that column. Finally, a small amount of error correction was applied to eliminate large discrepancies in the collected points do to background noise and missing line data which can occur when scanning over sharp points on a surface.

V. SURFACE RECONSTRUCTION

When considering what computational algorithm to use for surface reconstruction, two main choices were present depending on the priorities of the scan. The first and simplest algorithm to use is the Ball-Pivoting Algorithm[1] which functions by identifying starting seed triangles and rotating a sphere about the starting edges to locate new vertices and form new triangles. The key advantage of this approach is that it retains the original geometrical data of the point cloud without modifying the positions of the vertices. However, this method makes two key assumptions. First it assumes that the samples are distributed over the entire sample with a spacial frequency greater than or equal to some minimum value, and second it assumes that an estimate of the surface normal is available. While the second condition is easily satisfied by orienting all surface normals toward the known camera position, the first condition fails at high resolution and fails to generate a manifold surface on scanned surfaces with sharp edges. In high resolution scans, the error in the depth measurement of the scanner becomes larger than the inter-sample distance causing false seed triangles and overlapping surfaces. In surfaces with sharp protrusions the scan results in optical occlusion were the line laser was hidden from the camera by the cliff in the surface. This causes no points to be recorded while the line is not visible creating a large gap in the point cloud. By itself, the Ball-Pivoting Algorithm cannot close the mesh resulting in a large whole in the mesh. The issues with both of these conditions is visually demonstrated in Figure 3.

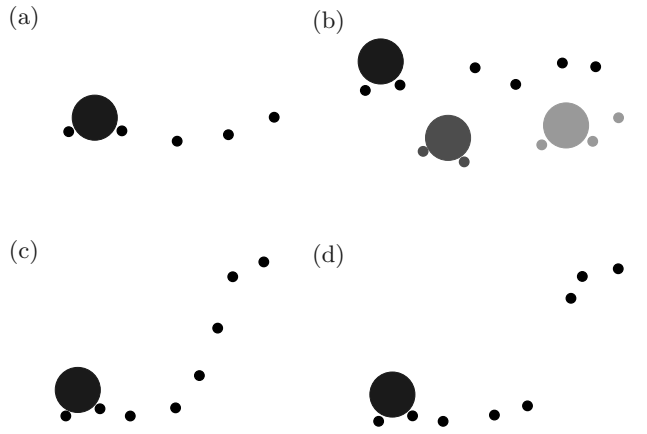

FIG. 3. Ball pivoting in the presence of good vs noisy data. (a) Error in depth measurement is negligible allowing for smooth reconstruction. (b) Significant error in depth measurement allows creates holes and double layering in mesh. (c) Smooth incline in surface resulting in smooth surface reconstruction. (d) Sharp protrusions in surface resulting in missing data points and holes.

While methods do exist to smooth out the point clouds and interpolate the missing data points, using both of these methods gives up the key advantage of the Ball-Pivoting Algorithm, which is that it retains the original geometrical data. As creating a watertight and accurate mesh held higher priority than aligning strictly to the specified data points, which were already demonstrated to contain errors, the second algorithm, Poisson Surface Reconstruction[2], was used instead. This algorithm computes a continuous 3D indicator function from the oriented point cloud normals, treating them as samples of a continuous vector field, to extract a smooth watertight mesh. Unlike the Ball-Pivoting Algorithm which constructs the mesh point by point, Poisson Surface Reconstruction takes then entire data set into account when constructing the mesh making it optimal for resolving noisy point cloud data while also ensuring a watertight mesh.

Thus, using this method, the point cloud was converted into a watertight mesh and exported to file with the Polygon File Format (PLY).

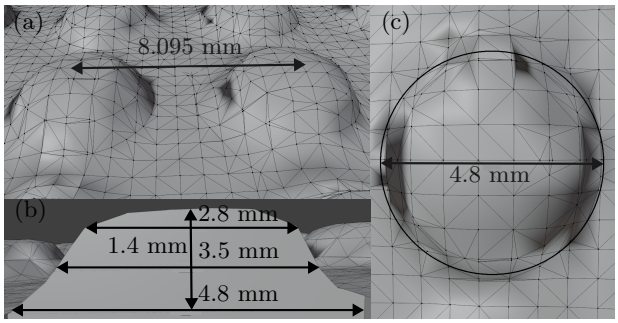

FIG. 4. Experimental measurements after 3D surface scan of 64 x 64 Lego baseplate taken in Blender. (a) Estimate of stud separation resulting in a 0.095 mm error off of the 8.0 mm actual separation. (b) Cross-sectional cut showcasing deformities in depth measurement resulting in near millimeter errors near the top of the stud. Additionally, a 0.3 mm error is noticed from the actual stud height of 1.7 mm (c) Top down image of surface scan demonstrating radial distance error of less than 1 mm with visible deformities.

VI. EXPERIMENTAL APPLICATION AND RESULTS

After the scanner was constructed, a high resolution scan of a 64 x 64 Lego baseplate was taken as this surface provided a consistent grid with known dimensions for accurate analysis along with providing bother steep cliffs and drops which can be challenging for some scanners. The scan was preformed with the camera set to 1080p resolution, the scan height was set to 200 mm and the step height set to 0.1 mm per line measurement. Lego studs are 4.8 mm in diameter, 1.7 mm tall and spaced 8.0 mm apart. The measurements as taken after importing the PLY file into Blender indicated a sloped diameter ranging from 4.8 mm at the bottom to 2.8 mm at the top as seen in Figure 4b. This is not surprising given that the smoothing nature of Poisson Surface Reconstruction tapers out sharp angles. Additionally, it was noted that the height of the stud was 0.3 mm shorter than the expected value of 1.7 mm as seen in Figure 4b, however this measurement was taken from the estimate of the surface at that stud which was higher than the surrounding surface. Finally, as shown in Figure 4a, the separation distance between the studs was measured to be roughly 8.095 mm which is very close to the expected value of 8.0 mm, and given the rough nature of this measurement as finding the center of the studs was difficult in Blender, this error is likely smaller.

Overall while deformities were observed, the majority of these was a result of the Poisson Surface Reconstruction which, while it can be countered by increasing the resolution, is beyond the resolution capabilities of the laser and webcam used in this setup. Additionally, while the depth measurements demonstrated small but significant error with a maximum magnitude of roughly half a millimeter, the horizontal and vertical distances were remained very accurate indicating that the likely cause of the error is in the error of the laser line thickness and measurement via the web camera.

VII. CONCLUSION

In this paper, an affordable 3D surface scanning setup capable of sub-millimeter resolution was described, and its results were shown proving its accuracy. While smoothing artifacts were noticed especially when the scanner encountered a sharp edge in the surface, the main cause was identified to be the Poisson Surface Reconstruction compensating for the missing data in during the transition where the web camera was unable to see the laser line. Additionally, while the horizontal and vertical measurements were shown to be very accurate, the depth measurements displayed errors up to a third of a millimeter. The root cause of both of these errors was determined to be mainly the imperfect thickness of the line laser along with the resolution of the web camera which caused uncertain errors in the pixel location of the laser line by the detection software, yielding slight errors in the depth calculations. Further experimentation and tuning of the light filters along with increased isolation of the device from ambient light may also help in increasing the accuracy of the scanners.

VIII. ACKNOWLEDGMENTS

The author would like to thank Terrence Smith for providing the dysfunctional Photon Mono X 6K resin printer used for this project. The author would also like to thank Owen Lueck for assisting in measuring the side profile of the scanned stud in Blender.

IX. REFERENCES

[1]F. Bernardini, J. Mittleman, H. Rushmeier, C. Silva, and G. Taubin, "The ball-pivoting algorithm for surface reconstruction," IEEE Transactions On Visualization and Computer Graphics **5**, 349–359 (1999).

[2]M. Kazhdan, M. Bolitho, and H. Hoppe, "Poisson surface reconstruction," in *Proceedings of the Fourth Eurographics Symposium on Geometry Processing*, SGP '06 (Eurographics Association, Goslar, Germany, 2006) pp. 61–70.